PORTFOLIO / 2026

FDE에 지원한 김정묵입니다.

Backend developer & AI engineer

Java와 Spring으로 API를 개발합니다. Python으로 추천·검색·이미지 처리 기능을 만들었고, 연구실에서는 LLM의 작업 분할과 오류를 분석했습니다.

프로젝트와 연구

01 **기웃**

공모전·프로필 API와 주간보고서 자동화

02 **질문 기반 이미지 분할**

질문에 맞는 객체를 찾아 픽셀 마스크로 변환

03 **대전광**

장소 추천·검색과 영수증·얼굴 이미지 처리

04 **LLM 에이전트 연구**

작업 분할 · 구조별 정확도와 비용 · 오류 분석과 복구

Java · Spring Boot · MySQL · JPA · Python
Docker · AWS · GitHub Actions · Notion · n8n

PROJECT 01 / GIUT

기웃

백엔드 개발 · 보고서 자동화



공모전·프로필·포트폴리오 API를 개발했습니다. DB 제약과 테스트로 중복 등록·롤백을 점검하고, Notion과 n8n으로 보고서 초안 생성과 활동 기록 반영을 자동화했습니다.

Java 21 / Spring Boot / MySQL / JUnit / Notion / n8n

담당한 기능과 해결할 문제

기웃은 대학생이 공모전을 찾고, 프로필과 포트폴리오로 팀원을 모집하는 서비스입니다. 공모전·프로필·포트폴리오 API, 인증·권한, 데이터 검증과 테스트를 담당했습니다.

AI 서버와 기웃의 분류 코드가 달랐고, 잘못된 데이터를 일괄 저장하지 않도록 검증이 필요했습니다. 팀원별 과제와 회의록을 모으는 반복 작업에는 Notion과 n8n을 사용했습니다.

02 / API 설계

외부 데이터 수신과 API 접근 범위

외부 공모전 수신과 일괄 롤백

AI 서버 전용 API 키 인증을 사용자 JWT와 분리했습니다. 한 요청에서 1~20건을 받고, DTO·모집 기간·대표 URL 수를 검증했습니다. 활동 유형과 분야 코드를 변환하되 모호한 분류는 거절했습니다. 공모전·URL·검증 이력 중 하나라도 저장에 실패하면 요청 전체를 롤백했습니다.

URL의 의미에 따른 중복 정책

공식 URL의 공고 간 중복은 막고, 일반 출처·모집 링크의 공유는 허용했습니다. 공식 URL에만 정규화된 URL 해시를 갖는 MySQL 생성 컬럼과 UNIQUE 제약을 적용했습니다. 한 공고 내부의 URL 중복은 별도로 막았고, 동시 등록의 제약 위반은 충돌 응답으로 바꿨습니다.

PROJECT 01 / GIUT / 02

기웃

백엔드 개발 · 보고서 자동화

외부 데이터 수신과 API 접근 범위 (계속)

프로필·포트폴리오와 공개 범위

프로필·포트폴리오·활동 이력의 조회·등록·수정 API를 개발했습니다. 공개 목록과 대표 항목을 관리하고, 유효기간이 있는 공유 링크에서 비회원이 볼 수 있는 범위를 나눴습니다. EntityGraph로 연관 조회의 N+1 문제를 개선했습니다. 요청·응답 예시와 오류 명세는 Swagger/OpenAPI로 공유했습니다.

인증·권한과 사용자 상태

카카오·애플의 인가 코드 교환과 서비스 JWT 발급을 연결하며 OAuth state, Apple nonce와 토큰 검증을 다뤘습니다. 학교 이메일 인증에는 사용자 단위 발송·오답 제한과 HTTP 429·Retry-After를 적용했습니다. 탈퇴 계정은 기존 JWT가 남아도 접근을 제한하고 신고·차단 관계를 프로필과 추천에 반영했습니다.

03 / 테스트

정상 요청·예외·동시 요청 테스트

Controller·Service·DB 테스트의 역할 분리

Controller 테스트로 요청 검증·인증·HTTP 응답을, Service 테스트로 업무 규칙을 확인했습니다. H2 기반 HTTP·인증 테스트와 격리된 MySQL 8.4 테스트를 구분하고, 실제 DB 동작이 중요한 URL 중복·일괄 저장·트랜잭션 롤백을 MySQL에서 점검했습니다.

스크랩 동시성 테스트 6개 통과

등록 전 존재 여부만 조회할 때의 경합을 다루기 위해 사용자 행 기준 비관적 잠금과 UNIQUE 제약을 적용했습니다. 팀·공모전 각각의 중복 등록, 등록 후 해제, 해제 후 등록을 MySQL 8.4에서 검증했습니다. 사용자 행 잠금이 같은 사용자의 다른 작업도 대기시킬 수 있다는 비용을 함께 고려했습니다.

조회 개선과 배포 전 테스트

공모전 상세의 스크랩 여부·수, 팀 목록과 활성 멤버 수 조회를 개선하고 URL 중복 확인을 일괄 조회하도록 구성했습니다. 게시 상태·조회 조건의 복합 인덱스와 적용용 SQL을 준비했습니다. GitHub Actions의 개발·운영 배포 흐름에는 이미지 게시·배포 전 clean test를 실행하도록 구성했습니다.

PROJECT 01 / GIUT / 03

기웃

백엔드 개발 · 보고서 자동화

04 / 보고서 초안

과제와 회의록을 모아 보고서 초안 생성

회차별 근거 자료 수집

예약 실행 흐름과 Notion 조회 노드를 구성해 현재 회차와 이전 회차를 찾았습니다. 회차 페이지의 팀원별 하위 데이터베이스와 data source를 동적으로 조회했고, 9회차 테스트에서 팀원 표 5개와 과제 13개를 읽는 것을 확인했습니다.

상태·본문·회의록을 연결한 초안

과제 제목·상태·본문과 회의록을 모으고 이전 회차의 과제를 담당자·제목 기준으로 비교했습니다. 일치 기록이 없다고 미수행으로 단정하지 않도록 했습니다. OpenAI 모델에 자료를 전달해 활동 요약 초안을 생성하되 출처를 보존하고, 모호하거나 충돌하는 내용을 임의로 확정하지 않도록 지침을 구성했습니다.

음성 회의록의 별도 처리 경로

음성 파일이 있는 회의록을 Notion에서 읽고 VITO 전사 상태를 조회해 결과를 활용하는 경로도 구성했습니다. 텍스트·음성의 통합 저장과 재사용, 전사 실패·시간 초과 처리는 추가 검증 대상으로 남겼습니다.

05 / 활동 기록 반영

활동 기록을 해당 회차의 팀원 표에 반영

기간과 대상 표의 동적 선택

별도 수동 테스트 흐름에서 개인 포트폴리오의 시작 전·기웃 기록을 조회했습니다. 게시일이 이전 회차 날짜의 다음 날부터 현재 회차 날짜까지인 기록을 고르고, 회차 페이지에서 김정묵의 하위 표와 data source를 찾아 고정된 표 ID에 의존하지 않도록 구성했습니다.

원본 링크를 기준으로 중복 확인

원본 본문을 Markdown으로 읽어 출처 링크와 함께 팀원 표의 개별 행으로 생성했습니다. 기존 행 본문의 원본 링크와 비교해 같은 기록의 재생성을 중단하도록 구성했습니다. 표 저장과 원본 상태 변경은 별도 단계이므로 중간 실패 시 상태가 달라질 수 있다는 점을 고려했습니다.

수동 테스트로 확인한 처리 결과

10회차에서 기록 3개를 생성하고 대상 행과 포트폴리오 원본 상태를 완료로 변경한 결과를 확인했습니다. 기존 세 기록을 다시 전달했을 때 중복을 감지하고 멈추는 테스트도 확인했습니다. 빈 표는 기존 행 조회를 건너뛰도록 분기했습니다.

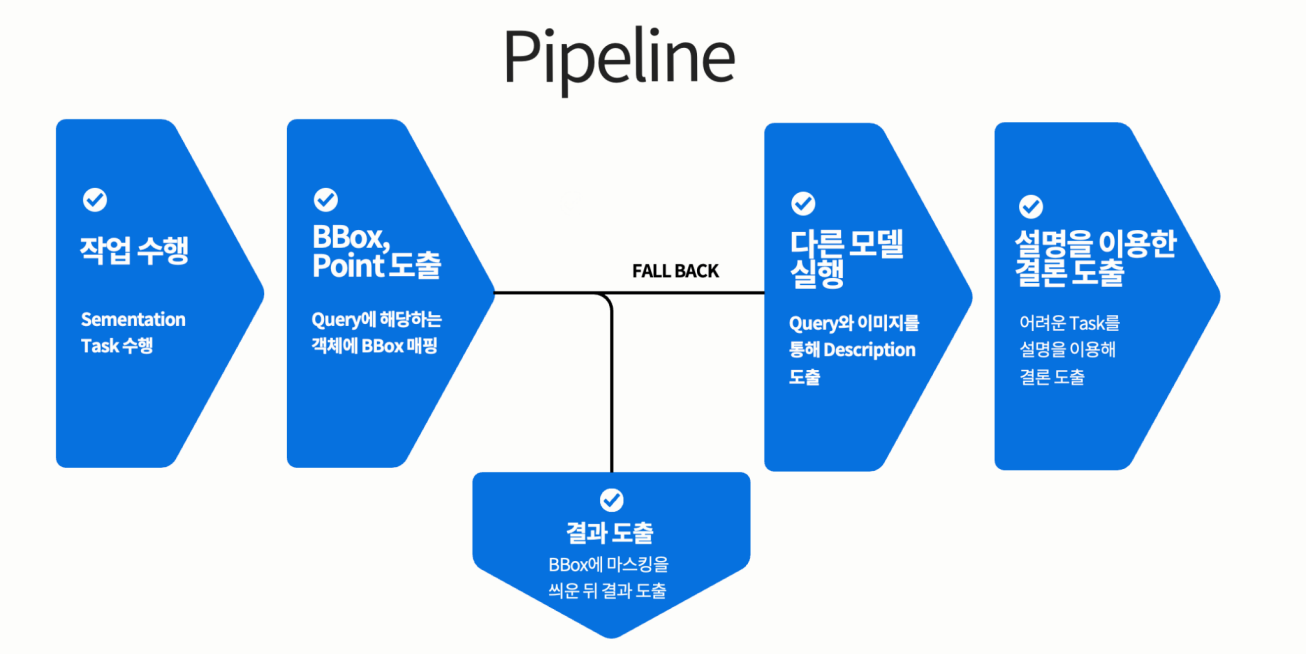
PROJECT 02 / VISION & LANGUAGE

질문 기반 이미지 분할

Competition 2 · 추론 파이프라인 구현 · 2026.08.10-14

이미지와 질문을 받아 대상의 위치를 찾고, SAM2.1로 마스크를 만들었습니다. 모델 출력이 잘리거나 유효한 좌표를 읽지 못하면 질문을 보강해 한 번 더 추론하도록 했습니다.

Python / Qwen-VL / SAM2.1 / RLE



01 / 과제와 담당 역할

질문의 답을 픽셀 영역으로 표시하는 과제

딥러닝 부트캠프의 Competition 2에서 추론 파이프라인을 맡았습니다. 이미지와 자연어 질문을 입력받아, 질문이 가리키는 객체를 찾고 제출용 마스크로 변환했습니다.

예를 들어 “지금 선두인 사람”을 찾으려면 사람의 위치뿐 아니라 경기 진행 방향과 선수 사이의 관계도 판단해야 했습니다. 다른 선수를 고른 경우에는 마스크 경계가 정교해도 오답이었습니다.

사전학습 모델을 연결하고 프롬프트·출력 파싱·좌표 변환·후처리를 구현했습니다. 이 파이프라인에서 모델 가중치를 학습시키지는 않았습니다.

PROJECT 02 / VISION & LANGUAGE / 02

질문 기반 이미지 분할

Competition 2 · 추론 파이프라인 구현 · 2026.08.10-14

02 / 시도한 검증 방법

검증 모델도 잘못된 대상을 정답으로 판단했습니다

5회 생성과 bbox 투표

같은 이미지와 질문으로 후보를 5번 생성하고, 비슷한 bbox를 IoU로 묶어 투표하는 방법을 검토했습니다. 한 번의 실행이 같은 객체를 중복 출력해도 여러 표로 세지 않도록 했습니다. 이 방식은 최종 노트북에는 포함하지 않았습니다.

제안 모델과 검증 모델의 분리

Qwen3가 제안한 대상과 좌표를 Qwen2.5가 검토하고 피드백하는 구조를 시험했습니다. 잘못된 proposal에도 PASS와 높은 점수를 주는 사례가 있었습니다. 기존 proposal을 보지 않고 독립적으로 대상을 찾거나, 판단 이유를 따로 검토하는 방법도 살폈습니다.

자기 점검 문구와 입력 크기 변경

프롬프트에 단계별 풀이와 Wait 문구를 넣고, 별도 verifier를 없앤 버전도 검토했습니다. 840 고정 resize를 바꾸는 실험에서는 함수 정의와 호출이 끊기는 오류도 다뤘습니다. 최종 노트북은 840×840 입력과 자기 점검을 유도하는 문구를 사용합니다.

03 / 최종 코드

위치 추론과 마스크 생성을 나눴습니다

Qwen3로 bbox와 내부 point 생성

Qwen3-VL-8B-Instruct에 이미지와 질문을 넣어 대상의 bbox와 point를 생성했습니다. <answer> 안의 JSON을 읽고 annotation 구조와 bbox 크기를 검사했습니다. point가 없으면 bbox 중앙을 사용하고, point가 bbox 밖에 있으면 내부로 제한했습니다.

출력 실패 시 질문 보강 후 한 번 재생성

첫 출력이 생성 상한에 도달했거나 파싱에서 유효한 결과를 얻지 못하면 Qwen2.5-VL-7B-Instruct로 이미지의 시각적 설명을 만들었습니다. 설명을 원래 질문에 붙여 Qwen3를 한 번 더 호출했습니다. Qwen2.5에는 숫자 좌표 대신 색·모양·위치 관계를 설명하도록 했습니다.

SAM2.1로 객체별 마스크 생성

SAM2.1 Hiera Large에 원본 이미지와 bbox·positive point를 전달했습니다. 여러 마스크 후보 중 SAM 점수가 가장 높은 것을 선택하고, 복수 객체의 마스크는 합집합으로 병합했습니다. SAM 점수는 정답 마스크와 비교해 측정한 IoU와 구분했습니다.

PROJECT 02 / VISION & LANGUAGE / 03

질문 기반 이미지 분할

Competition 2 · 추론 파이프라인 구현 · 2026.08.10-14

04 / 좌표와 오류 분석

대상 선택과 좌표·경계 오류를 따로 봤습니다

VLM 입력과 SAM 입력의 크기 구분

VLM에는 840×840 이미지를, SAM에는 원본 이미지를 사용했습니다. 출력 좌표의 최댓값이 1000 이하이면 정규화 좌표로 가정하고 원본 너비·높이에 비례해 변환했습니다. 이 판별은 코드의 휴리스틱이므로 모델의 좌표 규약을 따로 확인해야 합니다.

bbox·point·마스크를 겹쳐 확인

bbox는 빨간 사각형, point는 파란 점, 마스크는 초록 반투명 영역으로 표시했습니다. 잘못된 객체를 선택했는지, 좌표를 잘못 옮겼는지, 맞는 객체의 경계가 어긋났는지 나눠 살폈습니다.

형식 복구와 정답 검증의 구분

최종 코드의 복구 조건은 토큰 한도 도달과 파싱 실패입니다. 유효한 JSON에 다른 객체의 좌표가 들어 있는 경우는 이 조건만으로 잡을 수 없습니다. 5회 투표와 PASS/REVISE 반복도 최종 코드에는 넣지 않았습니다.

05 / 제출 처리

마스크를 RLE와 제출 CSV로 변환했습니다

열 우선 RLE 인코딩

마스크를 열 우선 순서로 펼쳐 1부터 시작하는 위치·길이 쌍으로 바꿨습니다. RLE 숫자 개수가 홀수라는 평가 오류를 다루며 쌍 구조를 확인했습니다. 빈 문자열, 결측값과 대회의 빈 마스크 표기는 별도로 확인할 항목으로 구분했습니다.

이미지 ID와 제출 ID 대응

입력 ID를 5자리로 맞춰 PNG와 JPG를 찾고, 제출 ID는 test_00000 형식으로 변환했습니다. CSV는 ID,label 열로 작성했습니다. 이미지 누락이나 처리 예외가 발생하면 빈 label을 기록하도록 했습니다.

06 / 배운 점

오류가 생긴 단계를 찾을 수 있어야 했습니다

검증 모델이 PASS를 주거나 여러 번 생성한 답이 일치해도, 질문과 다른 대상을 고르는 경우가 있었습니다. 이 경험을 통해 모델의 판단을 그대로 받아들이기보다, 어떤 대상을 골랐고 그 결과가 다음 단계로 어떻게 전달됐는지 확인해야 한다는 점을 배웠습니다.

VLM이 다른 대상을 선택한 경우와 SAM이 경계를 잘못 나눈 경우는 수정할 부분이 달랐습니다. bbox·point·마스크를 겹쳐 보며 오류를 나눠 살핀 덕분에, 모델을 추가하기 전에 각 단계의 입출력과 실패 조건을 정리하고 중간 결과를 확인할 수 있게 만드는 일이 중요하다고 느꼈습니다.

PROJECT 03 / DAEJEONGWANG

대전광

추천·검색·이미지 처리 서버

Spring이 전달한 장소 후보로 추천 점수를 계산하는 Python 서버입니다. 자연어 검색, 영수증 OCR과 얼굴 블러 기능을 개발하고, 모델 호출 실패 시 사용할 대체 경로를 마련했습니다.

Python / OpenAI / AWS S3 / Docker



01 / 서버 구성

추천 서버가 맡은 기능

대전광의 유사 장소·다음 장소 추천, 자연어 장소 검색, 영수증 분석과 얼굴 비식별화를 제공하는 Python 서버입니다. Spring이 장소 후보를 조회해 전달하면, 이 서버에서 추천 점수를 계산해 반환합니다.

요청과 응답은 Pydantic으로 검증했습니다. 추천 계산에는 NetworkX·NumPy·SciPy를, 이미지 처리에는 Tesseract를 사용하고 OpenAI API와 AWS S3를 연결했습니다.

02 / 장소 추천

장소 추천 점수와 대체 경로

유사 장소 추천

장소의 설명·태그·분류를 텍스트 프로필로 만들고 임베딩 코사인 유사도를 계산합니다. 분리 API는 의미 유사도 60%와 문맥 유사도 40%를 결합합니다. 임베딩이 없으면 태그 multi-hot 코사인 점수로 대체하고, 자신·방문 제외 장소를 제거한 뒤 반경 안의 후보를 정렬해 최대 5개를 반환합니다.

PROJECT 03 / DAEJEONGWANG / 02

대전광

추천·검색·이미지 처리 서버

장소 추천 점수와 대체 경로 (계속)

다음 장소 하이브리드 추천

거리 필터 후 혼잡도·방문자 수·앱 선택 횟수·거리·의미 유사도의 기본 점수, Personalized PageRank와 LLM 또는 규칙 기반 이동 적합성을 결합했습니다. 방문 이력이 있는 경로는 기본 점수 65%·정규화 PPR 10%·전이 점수 25%를 사용합니다. 최근 최대 4개 방문 이력과 현재 장소를 반영해 재추천을 제한합니다.

장애 시 대체 처리와 결과 추적

임베딩 실패는 태그 점수로, LLM 실패는 규칙 scorer로 대체했습니다. fallback의 문자열·정수 ID 불일치를 수정하고 점수 구성·출처·추천 이유를 응답과 로그에 남겼습니다. LLM은 적합성과 이유를 평가하고 최종 점수 결합은 코드가 수행합니다.

03 / 자연어 검색

한국어 검색어에서 핵심어 추출

핵심어 추출과 의미 검색

엄격한 JSON Schema로 핵심 명사·명사구를 최대 5개 추출하고 한국어 조사·요청 어투를 정규화했습니다. 태그 직접 매칭 후 임베딩이 활성화되면 의미 유사도로 재정렬하며, 추출·임베딩 실패 시 규칙 추출과 직접 매칭으로 대체합니다.
HTTP 응답은 최대 5개이며 전체 결과 수와 점수 출처를 별도로 제공합니다.

한국어 오탐과 예외 상황 보완

비슷한의 일부를 비로 인식해 비 오는 날 의도로 확장하던 문제와 한 글자 검색 의도 처리를 개선했습니다. 후보가 없으면 빈 검색 결과를 정상 반환하고, 원문·정규화 요청·핵심어·확장 태그·후보 수와 결과를 로그로 추적하도록 구성했습니다.

06 / API 명세

API 명세와 공통 오류 응답

14개 업무 API와 공통 오류 응답

추천·검색, 업로드·S3 영수증 분석, S3 접근 점검과 얼굴 모자이크 등 14개 업무 API를 구성했습니다. 한국어 Swagger/OpenAPI와 공통 error.code·message·request_id 형태를 정리하고 데이터 오류·객체 없음·용량 초과·형식 오류·외부 장애를 구분했습니다.

PROJECT 03 / DAEJEONGWANG / 03

대전광

추천·검색·이미지 처리 서버

04 / 영수증 분석

영수증 전처리와 구조화된 결과 반환

이미지 전처리와 OCR

JPEG·PNG·HEIC/HEIF의 방향·크기를 보정하고 영수증 영역을 잘라내고 원근을 보정합니다. Tesseract의 빠른 경로를 먼저 시도하고 부족하면 PSM과 이미지 변형을 바꿔 보완합니다. 상호명·주소·결제 일시·총액 등을 추출하고, 총액·공급가액·부가세의 관계와 누락 필드를 확인하도록 구성했습니다.

Vision 응답 구조와 오류 구분

영수증을 JPEG로 정규화해 GPT Vision에 전달하고 JSON Schema와 Pydantic으로 결과를 검증합니다. 양의 정수 총액이 없거나 JSON·응답이 잘못되면 별도로 처리하며, 키 설정 누락은 503·외부 호출 오류는 502·판독 실패는 422로 구분했습니다. 내부 구조화 결과와 외부 핵심 응답을 분리했습니다.

S3와 Spring의 동기 응답 계약

S3 객체 키·허용 prefix·용량·콘텐츠 유형을 검증하고 객체 없음·접근 실패 등을 구분했습니다. OCR 실행 전 읽기 점검과 객체 메타데이터 조회 API도 구성했습니다. 주 연동 경로는 콜백 방식에서 Spring 요청에 동기 HTTP 응답을 반환하는 방식으로 변경했으며 /api/v1/ocr와 /api/v1/ocr-gpt가 동일한 핵심 응답 계약을 사용합니다.

05 / 얼굴 비식별화

얼굴을 검출해 블러 처리

YuNet 검출과 좌표 환산

YuNet으로 얼굴을 검출하고 검출 영역에 기본 18% 여백을 더해 얼굴 크기에 비례한 Gaussian Blur를 적용합니다. EXIF 방향을 보정하고 검출용 축소 이미지의 좌표를 출력 크기로 환산하며 JPEG·PNG·WebP·HEIC/HEIF를 처리합니다.

얼굴이 없는 경우와 처리 자원 제어

얼굴이 없어도 정상 JPEG와 faceCount 0을 반환하도록 변경했습니다. 동시 작업과 이미지 처리 스레드를 각각 1개로 제한하고 모델을 재사용하며, 공유 검출기의 크기 변경·추론은 잠금으로 보호했습니다. 로컬 요청은 JPEG를 반환하고 S3 요청은 별도 UUID 객체로 저장해 원본과 결과를 분리했습니다.

RESEARCH / MULTI-AGENT LLM

작업 분할과 구조별 정확도·비용 비교

자연어처리 연구실에서 LLM 에이전트의 작업 분할과 실행 경로를 살폈습니다. 구조별 정확도·시간·토큰 사용량을 비교하고, 오류가 난 단계에 따라 다시 시도할 범위를 고민했습니다.

어떤 작업을 나눠야 호출 비용을 줄일 수 있을까?

에이전트를 추가하면 모델 호출과 결과 전달도 늘어납니다. 작업을 나눈 이점이 이 비용보다 큰지 확인하려고, 분할 단위와 실행 구조를 함께 검토했습니다.

입출력과 검증 기준이 분명한가

하위 작업이 무엇을 받아 무엇을 반환하는지, 결과를 따로 확인할 수 있는지 살폈습니다.

동시에 실행할 수 있는가

앞 단계의 결과가 필요한 작업과 독립적인 작업을 구분하고, 전달할 상태와 실패가 번지는 범위를 검토했습니다.

초기 실행을 보고 구조를 고를 수 있는가

처음 2~3단계의 진행 상황과 반복 탐색을 관찰해 단일·순차·병렬 구조를 선택하는 방법을 연구 질문으로 정리했습니다.

순차 구조의 정확도가 높았지만, 시간과 토큰도 늘었습니다.

실행 구조	Answer EM	평균 시간	평균 토큰
Single	53%	4.72초	2,050
Sequential	64%	13.51초	6,673
Parallel	55%	5.20초	6,270
Planner + Verifier	63%	38.63초	24,445

이 기록에서 Sequential의 Answer EM은 Single보다 11%p 높았습니다. 평균 시간은 약 2.9배, 토큰은 약 3.3배였습니다. Planner + Verifier는 더 많은 토큰을 사용했지만 Sequential보다 높은 EM을 얻지는 못했습니다.

RESEARCH / ERROR ANALYSIS

실패한 단계부터 살펴보기

계획이 맞아도 도구 인자를 틀리면 작업은 실패했습니다.

실행 기록에서 역할 선택과 출력 파싱을 살폈고, 도구를 사용하는 작업에서는 계획과 실제 행동 사이의 오류를 확인했습니다. 예를 들어 대상 ID나 수정할 필드·값을 잘못 구성하는 문제가 있었습니다.

오류가 난 단계부터 구분

계획 자체가 틀린 경우, 모델 출력의 형식이 깨진 경우, 관찰한 정보를 도구 인자로 옮기다 틀린 경우를 나눠 보았습니다.

전체 재실행 대신 부분 복구 검토

국소 오류에는 해당 단계의 수정·재시도, 계획 오류에는 재계획을 적용하는 방법을 검토했습니다. 검증을 추가하거나 맥락을 새로 구성하는 선택지도 비교 대상으로 정리했습니다.

다음 검증은 같은 예산에서 비교

오류 유형별 복구와 단순 재시도를 같은 호출·토큰 예산으로 비교하는 실험을 계획했습니다. 복구 정책이 성공률이나 비용을 개선했다는 결과는 아직 없습니다.

모델을 연결한 코드에서, 검증할 지점을 찾았습니다.

질문 기반 이미지 분할 · 단계별 오류

위치 추론, 좌표 변환, 마스크 생성과 제출 처리를 연결했습니다. 출력 잘림·파싱 실패에는 질문을 보강해 재추론했지만, 형식이 맞는 오답까지 검출하지는 못했습니다.

음성 전사 · 중간 결과의 재사용

전사 원문과 Qwen 후처리 결과를 따로 저장해, 음성을 다시 전사하지 않고 후처리할 수 있게 했습니다. 중간 결과를 보존할 때 재실행 범위를 줄일 수 있다는 연구 질문으로 이어졌습니다.

ABOUT / BACKEND & AI

개발할 때 신경 쓰는 것

직접 쓸 수 있는 기능을 만들고,
문제가 생기면
원인을 확인하려고 합니다.

김정묵 / Backend developer & AI engineer

기웃에서는 학교 이메일 인증과 스크랩의 동시 요청을 다뤘습니다. 정상 요청뿐 아니라 만료된 코드, 제한을 우회하는 요청, 중복 저장도 테스트했습니다.

VLM 실험에서는 검증 모델이 틀린 답에 PASS를 주는 경우를 봤습니다. 그래서 대상 선택, 좌표 변환, 분할 경계를 나눠 살폈습니다. 출력이 완성됐는지와 답이 맞는지는 따로 확인해야 했습니다.

요청과 오류 응답 정의

DTO 검증과 API 오류 명세를 함께 정리했습니다.

실제 DB에서 경합 검증

동시 요청과 롤백은 격리된 MySQL에서 확인했습니다.

자동화 결과에 출처 남기기

보고서 초안에 과제 상태와 원본 링크를 함께 남겼습니다.

수상과 활동

인공지능융합대학장상

서울시립대학교 · 딥러닝 경진대회

ALOC 회장

대외 프로그램과 파트너십 제안을 검토했습니다.

자연어이해와 생성

Transformer 구현 · LLM 실험 및 평가